Final Assignment - Multi Agent Large Data Processing Pipeline

Turn 97.4 MB of real Spotify reviews into a product recommendation supported by traceable evidence.

Overview

This individual capstone spans Classes 6 and 7 and is worth 30% of your course grade. Due: Oct 13, 2026 at 11:59 pm Pacific Time. Imagine you are advising Spotify at the end of this historical review window: where should the next quarter of product effort go—access, usability, playback, or billing/support? Build a pipeline that makes the answer inspectable. Bring a working 500-review enricher to Class 7; nothing is submitted after Class 6.

What You Are Submitting

- One public GitHub repository URL, submitted through the course submission portal linked in bCourses. Your README is the entry point for the grader.
- A working pipeline with distinct enrichment, verification, grouping, and memo-writing roles, coordinated by code. Roles may use the same model, but must have separate instructions, inputs, outputs, and saved evidence.
- An ingestion report and full pipeline run covering all 660,622 (or at least 100,000) original review IDs. Classify 660,609 (or at least 100,000) nonempty reviews; explicitly quarantine the empty texts. Preserve source values, IDs, and record statuses.
- The common topic and severity definitions, your examples and optional subtopics, output schema, prompts, hand-labeled 50-review golden set, evaluation results, and inspected failures.
- The enriched table, quarantine log, issue-to-review membership, aggregate table, reproducible ranking, and a short decision memo with issue IDs, review IDs, and supporting numbers.
- Run evidence: model IDs/settings, prompt versions, tokens, cost, runtime, retries, failures, and an interruption/resume recording with checkpoint snapshots. Also submit your runnable 100-review cost/runtime calculator, real pilot measurements, editable prices and offline replay command.
- A saved program that accepts an input CSV path and runs the stages without manual chat pasting, plus a command to regenerate ranking from saved outputs without model calls. Also required: a deployed web dashboard backed by a deployed backend and a database that stores the processed review data, aggregates, and analysis outputs. The backend must retrieve the saved data from the database and serve it to the dashboard. The dashboard must show overall metrics, issue rankings, and AI-generated overall

recommendations inferred from those analyzed metrics, with supporting numbers and traceable issue/review evidence. Include the live dashboard URL and any access instructions in the README; the grader must be able to inspect the saved results without rerunning paid model calls.

Grading Rubric: 10 Points

The course framework is deliverable quality (4), testing & evaluation (3), and working result (3). These criteria describe the evidence used to judge your project; they are not an automatic feature-counting formula. This assignment contributes 30% of the course grade.

- **Deliverable quality — 4 points**, one each: accessible code/setup and artifacts; clear architecture, shared schema and provenance; memo numbers linked to correct calculations and source evidence; coherent recommendation, alternatives and limitations. Cite evidence for each criterion.
- **Testing & evaluation — 3 points**, one each: 50 human labels, per-field comparisons and error analysis; independent verification plus planted-error and injection tests; a real 100-review cold/warm pilot, correct offline cost/runtime calculator, and demonstrated retry, spending and recovery controls (T3). Show actual outcomes, including failures.
- **Working result — 3 points**, one each: full ingestion, record coverage and successful classification; a runnable staged program with bounded calls, saved handoffs and demonstrated resume; reproducible baseline ranking and a usable deployed dashboard, connected through a backend to the processed data in a database, with grounded AI-generated recommendations. Diagrams alone do not demonstrate execution.

Partial credit follows the work and evidence provided. For qualitative criteria, complete evidence earns 1, materially incomplete evidence 0.5, and absent or contradictory evidence 0. The coverage point is proportional as specified under How We Check Your Work. Clearly mark incomplete runs; simulated outputs or a small demo are not the completed full-corpus run. There is no minimum label-accuracy target and no bonus for higher API spend, extra agents, or a particular provider. SQL, Jev and the named models are recommendations; choosing other tools carries no penalty by itself. Full-corpus classification is required. Low agreement can support strong evaluation credit when the method and error analysis are sound, while revealing limitations in the working result.

Make your README map each rubric category to evidence links. A reviewer should be able to inspect your conclusions without rerunning paid model calls, then rerun the deterministic ranking if needed.

Download the Dataset

- Download the complete Spotify dataset ZIP from Google Drive. The archive is about 42 MB; the full CSV inside is 97.4 MB. Download and unzip it before starting. Use Drive's Download button to retrieve all eleven files; its archive preview may omit the large CSV.
- spotify_reviews_18months.csv: 660,622 real reviews; 97,400,616 bytes. Ingest and profile the entire file, then classify every nonempty review through bounded model tasks.

- cost_100.csv is the fixed input for your required cost/runtime calculator; COST_CALCULATOR.md specifies that deliverable. checkpoint_500.csv and analysis_10000.csv are later development checkpoints. golden_50_to_label.csv has blank human-label columns. Also included: manifest.json, README.md, prepare_dataset.py, GRADING_CONTRACT.md and check_submission.py.
- Source: BwandoWando, 3.4 Million Spotify Google Store Reviews, version 2, on Kaggle. The publisher lists CC0: Public Domain. The original dataset contains 3,377,423 reviews. The course extract covers an 18-month window in May 2022–November 2023; see manifest.json for exact boundaries and checksums.

Before You Run: API Keys and GitHub Safety

For the suggested setup, you will need an OpenAI or Claude (Anthropic) API key, plus a Jev (TypeSafe) API key. You do not need both OpenAI and Claude. If you choose other providers or local models, configure only the credentials your implementation uses. Create keys in each provider's developer console and check API billing/credits before the 100-review pilot.

Keep real keys in a local .env file in your project root. Load them as environment variables through your program (for example, with a dotenv loader). Never hard-code keys or expose them in browser code, prompts, notebooks, screenshots, logs, or submitted artifacts.

Commit a blank .env.example with these names. Copy it to .env and fill in only the keys you use, on your own computer:

```
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
TYPESAFE_API_KEY=
```

TYPESAFE_API_KEY is the key used for Jev. Add other provider variable names to the example with empty values.

Before your first commit, add these lines to the repository's .gitignore and commit that file:

```
.env
.env.*
!.env.example
```

Before pushing: git check-ignore .env should print .env. Run git ls-files -- .env '.env.*'; only the blank .env.example may appear. Inspect staged changes for secrets. If .env was already tracked, use git rm --cached .env; ignore rules do not untrack it.

If a real key was committed or shared, revoke/rotate it immediately. Deleting the file does not erase Git history. Submit .env.example, .gitignore, and setup instructions—never .env or real keys. Offline calculator replay and saved-output checks must work without credentials.

Setup references: OpenAI key safety · Claude authentication · Jev quick start · GitHub ignore rules

Understand the Data Before Calling a Model

- review_id: original stable ID. review_text: original customer language, possibly with line breaks. review_rating: original 1–5 star rating; it is not your severity label.
- review_likes: helpful-vote count in the source snapshot. app_version: version associated with the review, often missing. review_timestamp: source timestamp with no specified timezone.
- Use SQL or ordinary code with a CSV parser that handles multiline text. Preserve IDs and source values; profile dates, ratings and missingness; find exact duplicate texts; look up cached results and queue pending IDs. Counting, filtering and sorting need no model. Filters organize work but must not remove required reviews.
- The full extract contains 13 empty review texts, 159,701 missing app versions, and no repeated review IDs. Verify these in your ingestion report. Keep a clear reason for any exclusion or quarantine; missing app version alone need not prevent classification.
- Preparation removed the original row index, author names, and author IDs, and renamed author_app_version to app_version. Retained field values are unchanged. Review text has not been translated or redacted; avoid repeating unnecessary personal details in the memo.
- The data has no account revenue, plan tier, confirmed cancellations, or complete customer population. Cancellation language is expressed intent, not observed churn. Describe self-selected, historical review evidence accurately; do not calculate revenue at risk or claim a causal retention improvement.
- cost_100.csv contains the first 100 rows of checkpoint_500.csv, nested inside analysis_10000.csv. The golden 50 is separate from all development files. The full final run includes every original text, including those 50 reviews. Never supply their human answer labels as model inputs. The manifest records the sampling method and checksums.
- Read all 97.4 MB with code. Enrichment requests contain at most 50 reviews each, with every returned ID validated and results saved after each batch. Account for all source IDs and disclose model-stage failures. A saved pipeline must run on an arbitrary supplied CSV; a chat transcript alone is insufficient.

Common Labels and Design Choices

- Use these eight exact primary topics: access (login/account access), usability (controls/navigation/ads), playback (playback failures/crashes/lag), downloads (offline/saved music), catalog (content/search/recommendations/lyrics), billing (price/charges/paywalls/premium entitlement), support (support contact/response), other. Custom subtopics are optional. GRADING_CONTRACT.md gives complete boundaries.
- Use the shared severity scale: 1 = no reported problem, praise, unclear content or a pure feature request; 2 = annoyance or generic criticism; 3 = degraded/restricted function with some use remaining; 4 = a core task clearly blocked; 5 = explicit serious

financial, privacy, or data harm. Add examples. Stars, angry language, or cancellation intent alone do not establish severity.

- Budget before scaling: build the required calculator and measure 100 reviews, then refresh it at 500 and 10,000 reviews. Estimate the full run including prompts, outputs, reasoning, retries, verification and fallbacks. Set spending, output, concurrency and fallback limits. The final input is all 660,622 reviews.
- Bring to Class 7: shared labels with examples, human-labeled golden 50, a completed 500-review run, evaluation results, and a working calculator with measured 100-review cost/time and full-run estimates. Include an offline replay command. Preserve every source review_id.
- The stage order makes inexpensive tools practical: prepare and deduplicate before classification, use fixed labels for repeatable decisions, then aggregate before memo writing. SQL/code and Jev are suggested tools, not required technologies. Choose your implementation and explain its measured quality, cost and runtime. Consider small models for bounded language tasks and cap any stronger-model fallbacks.
- A fixed sequence is acceptable. Explain where code chooses the next step and where a model exercises judgment. Distinct roles and inspectable handoffs make this a multi-agent pipeline; unrestricted autonomy or different providers are not required.

Required: Build a 100-Review Cost and Runtime Calculator

Create a command, notebook or small local interface. Run the actual pipeline on cost_100.csv unchanged, using an empty result cache and one worker. Include enrichment, a declared verification sample, grouping, ranking and memo generation. Save all 100 statuses and every attempted call, including failures, retries and fallbacks. The complete specification and file checklist are in COST_CALCULATOR.md in the dataset ZIP.

Measure and replay: report cost and end-to-end wall-clock time for the real 100-review run. Repeat with saved results and demonstrate zero new enrichment calls under unchanged settings; disclose any downstream calls. Provide a default offline mode that recomputes costs from saved usage and editable rates without an API key. Paid pilot execution must be an explicit separate command.

Show by stage: provider, exact model/effort, prompt/schema version, batch size, worker count, completed/failed records, unique texts, cache hits, requests/attempts, usage, price units, currency and dated source links. Report cost per 1,000 inputs, cost per completed record, throughput and cold/warm time. Separate API spend from local-compute estimates and unknown costs.

Calculate each billing item as billed units × price per unit. Use mutually exclusive input, cached-input and output categories; do not double-count reasoning already billed as output. Apply only the processing tier actually used. Keep raw usage and clearly label missing usage or estimated charges. Summed call durations are not wall-clock time when calls overlap.

Project the full run: 660,622 rows, 660,609 nonempty classifications and 13 empty-text quarantines. Valid exact-text reuse reduces first-pass work to 484,189 distinct texts. Show a no-reuse comparison, stage-specific verification/fallback rates, fixed overhead once, and base/conservative cost and time scenarios. Do not multiply one memo's cost by every review. The first 100 are an initial estimate; refresh it after 500 and 10,000 reviews before scaling.

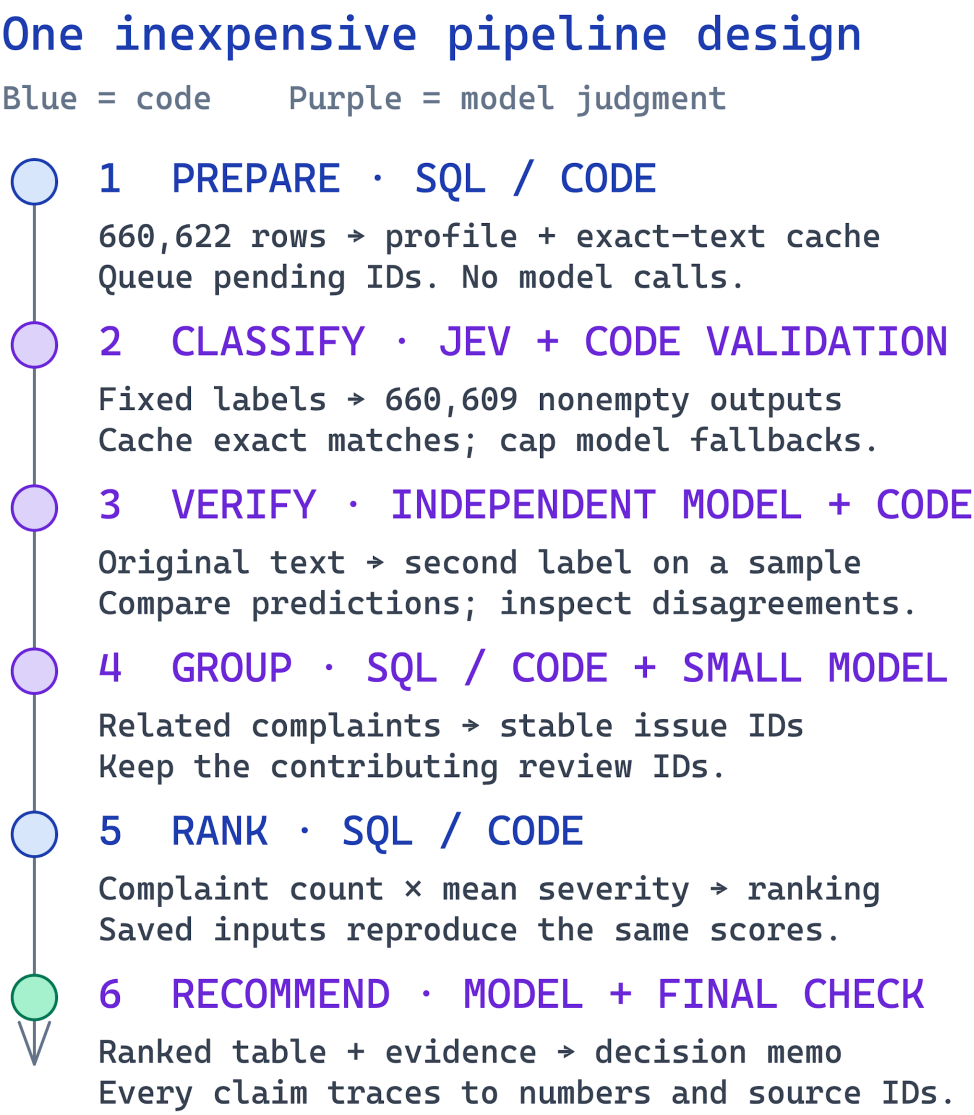
Set controls in the calculator: editable spending limit, output-token cap, maximum workers and fallback fraction. Show when a projected scenario exceeds the budget. Save pilot_records.jsonl, pilot_calls.jsonl, rates.csv, usage.csv (or equivalent structured files), report.md/html and replay instructions under cost/. The instructor must be able to check arithmetic from this evidence without a paid rerun.

Batching has three meanings: CSV chunks control memory; multi-review requests amortize instructions but stay at 50 reviews or fewer and within token limits; eligible provider Batch APIs may lower price with different turnaround. Measure the tier you actually use. Saved-result caching and provider prompt caching are separate mechanisms. Reuse exact results only under the same configuration and preserve every original ID.

Evaluate the Labels Fairly

- Hand-label all 50 golden reviews before running the final evaluation. Keep their expected labels out of prompts, examples, routing thresholds, and issue-discovery inputs. Use development records for prompt tuning. If golden-set results influence revisions, disclose that and add fresh held-out cases for the final check.
- Report topic and intent agreement, severity exact agreement and mean absolute error, and the number of ambiguous cases. Show per-topic counts and a confusion table where useful. Fifty cases provide a small diagnostic sample, not a precise population-accuracy estimate.
- For sentiment, predeclare a numeric tolerance or report mean absolute error. For evidence_quote, check exact substring membership in code and inspect whether the quote supports the label. For entities, inspect unsupported additions. Evaluate needs_review as a prediction, not an automatic correctness guarantee.
- Save expected labels, predictions, pass/fail details, and concrete disagreements. Do not infer sentiment or severity from stars alone. Explain ambiguous or unsupported-language handling, and do not silently drop hard cases to improve the score.

Six Steps with Suggested Tools

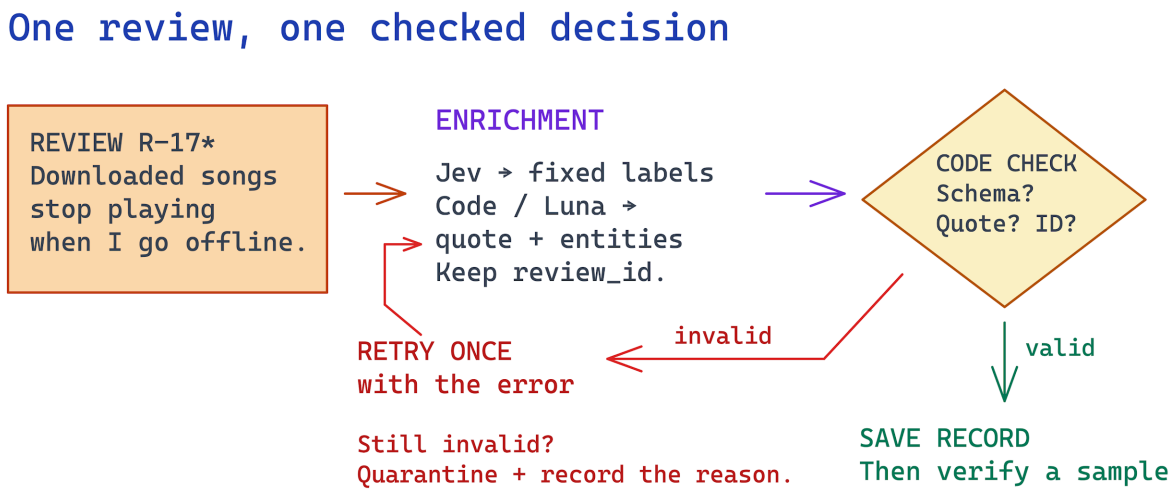


1. Prepare — read all 660,622 rows, preserve IDs and source values, profile quality, find exact duplicate texts and queue pending work. SQL or ordinary code can do this without a model. Output: ingestion report, manifest and pending IDs.
2. Classify — fixed topic, intent and severity choices suit Jev or another classifier/model you select. Use code for schema validation and deterministic extraction where it works; use a small model for remaining language tasks. Classify all 660,609 nonempty reviews in requests of at most 50. Retry invalid output once, then save or quarantine. The 13 empty texts use empty_review_text. Preserve direct cache provenance.
3. Verify — a separate model task re-labels a declared random sample without seeing the first prediction. Code compares results; investigate disagreements. Include a deliberately wrong label in a separate test copy. Output: verifier predictions and a disagreement report.

4. Group — save stable issue membership for complaint/cancellation records; SQL/code is a convenient option. A small model can suggest issue names from bounded examples. Keep every contributing review ID and save the accepted mapping so later ranking needs no model.
5. Rank — SQL/code computes priority(issue) = complaint_count(issue) × mean_severity(issue), exactly equal to severity_sum. Include complaint/cancellation records; count each review once per issue. Sort by descending score, then ascending issue ID. Save membership and scores; an alternative business ranking may be additional.
6. Recommend — the memo task receives only the saved aggregates and a bounded evidence pack. It recommends a priority, compares alternatives, and cites issue IDs and numbers. Code checks cited IDs and numerical claims; a person inspects the final argument.

- Each stage has a defined input, output, owner, failure behavior, and stop condition. The orchestrator handles dispatch, state, validation, budgets, and logs. The model reads messy language; code owns record accounting and arithmetic. Show both responsibilities in your architecture.

Example: One Review Becomes a Checked Record



Illustrative review, not a dataset quote: “Downloaded songs stop playing when I go offline.” A candidate topic is downloads and intent is complaint. Apply the shared severity definitions to the reported impact and flag missing context.
Every completed record has review_id, source_sha256, status, topic, intent, sentiment (−1 to 1), severity (integer 1–5), entities, evidence_quote (exact source substring), needs_review, and label_config. Allowed intents, in precedence order: cancellation, complaint, request, praise, unclear. GRADING_CONTRACT.md defines source hashing and the exact export schema. Check required keys, types, allowed labels, numeric ranges, exact source ID, and quote membership. A valid record can still be wrong: a real quote may not support the chosen label. Route unclear evidence for review and preserve the reason.

For difficult cases, give a small model one original review, relevant definitions and a specific question. Use Sonnet 5 only within a declared fallback quota when its measured benefit justifies the cost. Require a structured result and exact quote. A fallback cannot invent account information or change the taxonomy.
Keep statuses explicit: pending, completed, or quarantined with a reason and attempt count. Limit invalid-output retries to one. Treat rate limits and temporary API errors with bounded backoff; stop when the time or spend cap is reached and save progress.

Tips: Jev as One Classification Option

- Use TypeSafe's Choice for topic and intent: provide option names plus clear descriptions, including other/unclear. Several questions about the same review can share one call. Jev returns choices and probabilities; code maps them into your schema. See the Choice documentation for the current request shape.
- Use Score for an ordered scale only after defining each level. Its result is a position starting at 0 and may be fractional. For five sentiment levels, sentiment = score / 2 − 1 maps 0–4 to −1–1. For integer severity, a Choice over five defined labels is simpler; if using Score, document your probability-to-level conversion. See Score documentation.
- Use Noul for one yes/no proposition, such as whether the review explicitly expresses cancellation intent. It returns a probability from 0 to 1, not a boolean or a severity score. Choose thresholds using development examples and send uncertain cases for review. Treat confidence as a signal to evaluate, not a calibrated guarantee. See Noul documentation.
- Keep extraction inexpensive: a short review's complete original text can be its evidence quote, and code can match explicit feature terms. Evaluate whether the quote supports the label. Use GPT-6 Luna or another small model where language judgment is needed. Compare the complete enrichment output on the golden set before choosing the setup.

Tips: Inexpensive Models for Individual Tasks

For small language tasks, consider GPT-6 Luna, Google's Gemini Flash-Lite, Claude Haiku, suitable DeepSeek/Qwen models, or a local model. Sonnet 5 is an option for bounded difficult tasks or a memo when justified. Record exact versions, current prices and access. You do not need to pay to try every provider.

- Use the lowest thinking effort that passes your evaluations. GPT-6 Luna supports reasoning.effort = none or low; its default is not the cheapest setting. Gemini Flash-Lite and local runtimes have model-specific controls. Check the official documentation, cap output tokens and ask for concise structured answers. Local inference can have zero API charges while still using hardware, memory and time.
- Separate the verifier from the enricher's answer: give it the original text and rubric first, then compare in code. For grouping, send a bounded set of examples with IDs. For memo writing, send the ranked table and selected evidence—not the whole raw dataset. Reject unsupported new IDs, counts, or customer facts.

- Give every role a bounded task, input, output schema and stop rule. Use structured outputs where supported and handle incomplete/refused responses. Version prompts and model/effort settings. Keep a model allowlist and a capped fallback fraction; estimate cost from actual usage and current rates, including failures and verification.

Save Your Results

Keep a stable run ID and a manifest tying source checksum, code version, prompts, model IDs, settings and outputs together. Your internal organization is flexible. Also provide the standardized grading/ export described under How We Check Your Work so the instructor can check all students consistently.

- data_manifest.json and ingestion_report.json: exact source identity, full-file profile and failure accounting. enriched.jsonl or .csv: original IDs and validated predictions. quarantine.jsonl: unresolved records with reasons and attempts. The standardized export combines final statuses in records.jsonl.
- evals/: the human-labeled golden set, model predictions, per-case and per-field results, independent verifier results, planted-error and injection cases, and reproducibility/recovery evidence. Mark synthetic tests and exclude them from business aggregates.
- issues.csv or .json: stable issue IDs and member review IDs. aggregates.csv and ranking.csv: counts, mean severity, score, and tie-break order. Save group membership so reranking is deterministic.
- run_log.jsonl and run_summary.json: stage timing, statuses, attempts, failures, usage, actual charges or clearly labeled estimates, spending limits and resume evidence. cost/: calculator, pilot_records.jsonl, pilot_calls.jsonl, usage/rates, measured-100 report, projections and offline replay instructions. Do not include API keys.
- memo.md or .pdf: concise recommendation with traceable evidence. README.md: setup, run commands, design choices, results, and limitations. Include dependency versions, prompts, a blank .env.example and a .gitignore that excludes .env and other real credentials.
- Cache valid exact text under unchanged model, effort, prompt and schema settings. There are 484,189 distinct nonempty texts. Keep all original IDs and direct cache_source_id provenance; do not merge business counts. Save results/statuses atomically after each batch so workers cannot overwrite or double-count work. Pending and quarantined rows are not completed classifications.
- You do not need to commit the 97.4 MB raw CSV. Link the supplied dataset, record its checksum, and document where to place it. Keep all code, evaluation evidence, and saved analysis outputs accessible to the grader; use release assets or shared artifact links for large outputs.

How We Check Your Work

Add one grading/ folder: run.json, ingestion.json, records.jsonl, membership.csv, ranking.csv, claims.csv, calls.jsonl, checkpoint_before.json, and checkpoint_after.json. Large JSONL files may be gzip-compressed. GRADING_CONTRACT.md in the dataset ZIP is the required format; check_submission.py is the zero-API self-check.

Coverage: the checker compares every source ID and original row hash, reports missing/duplicate/foreign rows, and separates successful classifications from quarantines. It checks exact quote membership and declared exact-text cache reuse. Hashes and logs support consistency checks; they do not alone prove execution or correct interpretation.

The one coverage point is: 0.2 × exact full-file profile match + 0.2 × valid completed-or-quarantined rows / 660,622 + 0.6 × valid classified nonempty rows / 660,609. Duplicate IDs and invalid records do not count. The instructor verifies evidence before releasing any grade.

Analysis: the checker recomputes every issue's count, severity sum, mean and baseline ranking from saved records and membership. It compares claims.csv numbers with those calculations. Means use six decimals with half-up rounding. Humans check whether the memo includes all material claims and supports its recommendation.

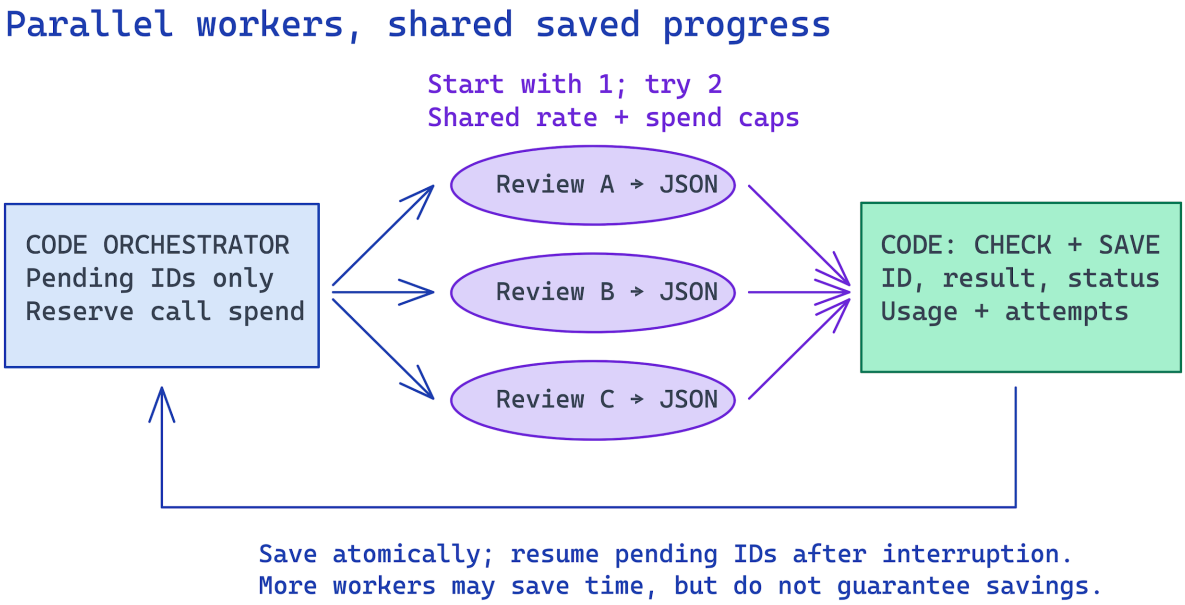
Label quality: the instructor compares saved predictions against one fixed private sample reviewed by instructors. Topic/intent agreement, macro F1 and severity error are diagnostics under the 4/3/3 rubric. Missing or quarantined benchmark predictions count as incorrect. Ambiguous cases can have several accepted labels; reference errors are corrected consistently for the whole cohort.

Execution: submit successful/failed call outcomes, exact model and label_config, actual usage when available, input review IDs, and initial/resume phases. The enrichment limit is 50 reviews per request. The recording and checkpoints must show saved work, interruption, then new completed work without paying to relabel completed IDs under unchanged settings. A small fresh-input demonstration may be requested after submission.

Apply common meanings: choose the highest-severity specific problem, then the first one mentioned on a tie. For praise, use the first specific praised feature. A paid-plan mention alone is not billing. Cancellation takes precedence over complaint, then request, praise, unclear. Bare boycott slogans are unclear unless a product complaint or personal departure is expressed. GRADING_CONTRACT.md includes all definitions and examples.

Cost grading: instructors check your saved 100 pilot IDs, usage/rate arithmetic and offline replay without paid calls. Doubling API rates with usage fixed must double API spend while leaving measured time and local costs unchanged. Changing projected volume must not change measured pilot results. T3 requires the real cold/warm pilot, a correct calculator and demonstrated retry/spending/recovery controls; complete = 1, materially incomplete = 0.5, absent/contradictory = 0. The main checker does not certify bills or the calculator.

Suggested Workflow and Parallel Processing



- Start small: inspect data with SQL/code, define labels, hand-label the golden set, and test one review. Build the calculator and measure the fixed 100-review pilot before expanding to 500, then 10,000. The full required run still covers every source ID.
- Parallelize carefully: start with one worker, try two, and increase only within measured capacity. Share global request/token limits and one spend ledger. Reserve estimated worst-case in-flight cost before dispatch and stop admitting work when spent plus reserved plus the next reservation exceeds your cap. Use bounded retries with backoff and jitter; reconcile timeout charges. Provider controls can help, but delayed billing limits any local guarantee.
- Compare the same reviews under the same cache conditions. Extra cold experiments cost money; label modeled speedups if you do not run them. More workers may reduce elapsed time but do not inherently lower cost; rate limits, retries and local memory contention can make them slower. Save each stage's handoff, reuse completed work, and rerun affected downstream stages when inputs change.
- Test awkward paths: malformed output, missing text, ambiguous labels, a temporary API failure, an injected instruction in a synthetic review, a deliberately wrong label, and an interrupted run. Show actual outcomes and the record counts before and after recovery.
- Review the top issues and their source examples, write the memo, and rerun ranking from saved outputs. Compare identical inputs to identical outputs. Finish the README and check every evidence link before submission.

Starter Prompt for Your AI Assistant

Help me build the Final Assignment pipeline for the supplied Spotify dataset. Read GRADING_CONTRACT.md and COST_CALCULATOR.md. Arrange preparation and

deduplication before fixed-label classification, and aggregate before memo writing. Consider SQL/code and Jev for the early steps, but choose tools based on measured quality, cost and runtime. Build a runnable cost/runtime calculator with a real cold/warm 100-review pilot, saved usage, editable dated rates, full-run scenarios and default offline replay. Paid execution must be an explicit command. Use cheap low/no-thinking models where supported for bounded language tasks; cap stronger-model fallbacks. Measure 100, then 500, then 10,000 reviews before the full run. Ingest all 660,622 rows, classify all 660,609 nonempty texts, and quarantine 13 empty texts. Preserve common labels, original IDs, hashes, quotes and cache provenance. Keep golden answers out of inputs. Limit requests to 50 reviews; begin with one worker and share rate limits/spend reservations across workers. Save results atomically, implement resume, export grading/ and cost/, and run the checker. Add independent verification, injection/planted-error tests and deterministic ranking. Store processed data and analysis outputs in a database, deploy a backend to serve them, and deploy a dashboard that shows overall metrics, issue rankings, and AI-generated recommendations grounded in those metrics and source evidence. Add the live URL and access instructions to the README. Explain results; never invent executions, usage, costs, revenue or churn.

Evidence Required in the README

- A compact results summary: source/completed/quarantined counts, exact-text cache reuse, golden agreement, verifier disagreements and full-run cost/time. Also link the measured 100-review cold/warm report, calculator and offline replay, dated rates and projection assumptions. Separate measured results from estimates and accounted-for rows from completed classifications.
- Your architecture diagram: show the six stages, code/model boundaries, each role's input/output, stored artifacts, and stop/retry paths. Show how the database, backend, and deployed dashboard connect. Explain why each model call is needed and what code does instead.
- One real review traced through source ID → enrichment → verification → issue membership → ranking → memo claim. Include one failed or ambiguous case with the recorded handling decision.
- The complete golden-set comparison and system checks: expected labels, model predictions, error analysis, independent-verifier procedure, injection/planted-error outcomes, and interruption/resume evidence.
- The required baseline, aggregation rules, stable tie-break and scope. For additional trend analysis, compare complaint shares in comparable periods and report denominators; the first and last calendar months are partial. Explain incomplete classifications, missing data and review bias.
- A concise decision memo: priority, supporting counts and severity, alternatives, representative review IDs and limitations. Cite claims.csv claim IDs for material issue-level numbers. Show that the prose matches saved calculations from the full run and discloses unresolved classifications.

Submission Checklist

1. Create or update your public GitHub repository. Include source code, dependency/setup instructions, role prompts, label definitions, saved results, evals, the memo, and the README. Save the final run and its manifest before publishing the evidence.
- Test the live dashboard and confirm that its backend retrieves the processed data from the database, that displayed metrics match saved calculations, and that AI-generated recommendations cite supporting metrics and evidence. Include the dashboard URL and any access instructions in the README. Test setup, offline calculator replay and ranking from saved outputs in a clean environment. A reviewer must be able to inspect and recalculate evidence without an API key. Opening/importing the calculator must not make paid calls; actual model execution uses a separate, explicit command.
2. Open the repository and evidence links signed out. Confirm that the grader can read or download them. Confirm .env and real keys are absent from tracked files, Git history and artifacts; include only a blank .env.example and the .gitignore rules. Revoke/rotate any exposed key. Remove unnecessary personal details. Keep the supplied raw-data source link and checksum in the README.
3. Identify the final full-corpus run, downloadable grading export, quarantined records and known limitations in the README. Honest failure accounting earns partial credit; nonempty quarantines still reduce the successfully classified fraction.
4. Submit your public GitHub repository URL through the course submission portal linked in bCourses. Select the final assignment when available. The deadline is shown in the Overview; the repository and linked artifacts are the single deliverable.

Learning Focus

Show that you can turn messy text into an auditable business argument. The key decisions are what to ask a model, how to constrain and evaluate the answer, how agents exchange evidence, and where code should control the process. Be ready to explain your system and one failure in your own words. API spend and fluent prose do not replace that understanding.

Definition of Done

- The entire 660,622-row source file is ingested, profiled and accounted for with original IDs and exact source hashes (or at least 100,000).
- All 660,609 nonempty reviews have valid classifications (or at least 100,000); empty texts are explicitly quarantined. Other failures remain visible and count as incomplete classification. Costs and retries are recorded.
- Distinct enrichment, verification, grouping, and memo tasks exchange saved, inspectable artifacts under code control.
- Golden-set and system evaluations are saved with actual outcomes, failure analysis and interruption/resume evidence. The 100-review cost/runtime calculator replays saved measurements without paid calls; full-run estimates and controls are documented.

- The ranking can be regenerated from saved issue membership and enriched records without a model call, with the same scores and order.
- The memo answers the product-priority question and every material numerical claim traces to a saved calculation and source evidence. The deployed dashboard is accessible to the grader and uses a deployed backend to serve processed data stored in a database. It displays overall metrics, issue rankings, and AI-generated overall recommendations grounded in those metrics, with traceable supporting evidence.

Single Deliverable

- One public GitHub repository URL through the submission portal in bCourses. Put the explanation, live dashboard URL, access instructions, and evidence links in the README, with runnable pipeline/dashboard/backend code, database setup instructions, saved analysis outputs, evals, and the final memo available from that repository.

Scope and Helpful References

- Required: the full-corpus pipeline, shared labels, distinct roles, bounded requests, saved outputs, validation/recovery, evaluations, deterministic ranking, grading export, grounded memo, a deployed dashboard and backend serving processed data from a database with overall metrics and AI-generated recommendations, and the measured 100-review cost/runtime calculator with offline replay. The 100, 500 and 10,000 files are development checkpoints.
- Optional: extra models or agents, a different grouping method, or an additional business ranking. The deployed dashboard, backend, and database are required; the hosting provider and technology stack are your choice. Use any language/framework you can explain and rerun. Training a model or buying a GPU is unnecessary.
- Official integration references: TypeSafe Choice, Score, and Noul; Anthropic's Claude Sonnet 5 overview; OpenAI's GPT-6 Luna model page. Check current API access and prices before running. The named tools are recommendations, and equivalent tools are acceptable when justified and evaluated.
- The ZIP includes the dataset, fixed development samples, golden template, README, manifest, preparation script, grading contract, deterministic checker and calculator specification. Cite exact model versions and settings. Numerical cost examples remain illustrations until validated by your pilot.